Roadmap: Building an LLM from Scratch to a Coding Agent

Nov 5, 2026 · @Giáo sư Thịnh

Overview

The path is pretraining, then chat post-training, then code specialization, then tool use and agent loops. This matches how modern models are built.

Building each stage from scratch teaches the mechanics, but a hobby-scale model will not be a good coding assistant or agent. For stages 3 and 4, pair your own small model with a strong open pretrained model so you can see what working behavior looks like.

Before you start (complete beginner setup)

You do not need a PhD or a lab. You need basic Python, a little math, and patience. Spend your first week on the items below, and do not skip them: every later stage assumes them.

1. Learn enough Python

You should be comfortable with variables, functions, loops, lists, dictionaries, classes, and reading error messages. If not, work through a free beginner course first (for example the official Python tutorial at python.org, or "Automate the Boring Stuff with Python"). A good test: can you write a function that reads a text file and counts how often each word appears? If yes, you are ready.

2. Learn the three math ideas that matter

You do not need advanced math. You need intuition for three things:

1. **Vectors and matrices:** a vector is a list of numbers, a matrix is a grid of numbers. A neural network is mostly multiplying vectors by matrices. Watch 3Blue1Brown's "Essence of Linear Algebra" series on YouTube (about 3 hours).
2. **Derivatives and the chain rule:** a derivative tells you how much the output changes when you nudge an input. Training a model means nudging numbers to make a score better, and the chain rule is how that nudge is passed backwards through many layers. Watch 3Blue1Brown's "Essence of Calculus" for the idea.

3. **Probability basics:** a model outputs the probability of each possible next token, and you pick one. Know what a probability distribution is, and why probabilities add up to 1.

3. Install your tools

1. Install **Python 3.10 or newer** (python.org, or Anaconda).
2. Install **PyTorch** from pytorch.org. The site gives you the exact install command for your system. PyTorch is the library that does the matrix math and the backpropagation for you.
3. Install **VS Code** (a free code editor) or use **Jupyter notebooks** (code cells you run one at a time, great for experiments).
4. Install **Git** and make a free **GitHub** account, so you can download other people's code and save your own.
5. Create a **virtual environment** per project (`python -m venv .venv`) so libraries from different projects do not clash.
6. Make a free **Weights & Biases** or **TensorBoard** account later, when you want to draw loss curves.

4. Decide where to run the code

Training is slow on a normal CPU. You want a GPU (a graphics card, which is very good at matrix math). Four options, cheapest first:

Option	Cost	Good for
Google Colab (free tier)	Free, limited hours	Stages 0-1, tiny models, learning
Kaggle notebooks	Free, about 30 GPU hours per week	Same as above, more predictable limits
Rented cloud GPU (Lambda, RunPod, Vast.ai)	Roughly $0.30-3 per hour	Stages 1-4, bigger runs
Your own NVIDIA GPU with 8-24 GB memory	Hardware cost	Everything up to small fine-tuning

Start free. Rent a GPU only when a run takes too long on free hardware. Prices and free limits change, so check the current numbers on each site.

5. Work habits that save you weeks

- Type the code yourself while watching a video. Do not copy-paste. Pause, predict what comes next, then check.
- When something breaks, read the last line of the error message first. Then search it.
- Change one thing at a time and write down what happened in a notes file.
- Print the shape of your tensors (`x.shape`) constantly. Most beginner bugs are shape mistakes.

Stage 0: Foundations (1-2 weeks)

What this stage is for: understand how a neural network learns, before you build a big one. If you skip this, later stages will feel like magic. If you do it, they will feel like bookkeeping.

Key idea: a neural network is a big pile of numbers (called *weights*). To train it, you (1) feed it an example, (2) measure how wrong its answer is (the *loss*), (3) figure out which direction to nudge each weight to reduce the loss (*backpropagation*), and (4) nudge them a tiny bit (*gradient descent*). Repeat millions of times.

Step 1: micrograd (about 2.5 hours of video, plus your own coding)

1. Search YouTube for "The spelled-out intro to neural networks and backpropagation: building micrograd" by Andrej Karpathy.
2. Open an empty notebook next to the video. Pause often and type every line yourself.
3. You will build a tiny engine that records every math operation, then walks backwards to compute how each number affected the result. That is backpropagation.
4. **Done when:** you can explain in your own words what `loss.backward()` does, and your tiny network learns a toy problem.

Step 2: makemore series (5 videos, about 1 week)

makemore teaches a model to invent new names by predicting the next letter. It is the same task as ChatGPT, just tiny.

1. **Part 1 (bigrams):** predict the next letter from only the previous letter. You learn what a probability distribution and a loss are.
2. **Part 2 (MLP):** use a small neural network and *embeddings* (each letter becomes a vector of numbers).

3. **Part 3 (activations and batchnorm):** why training sometimes fails, and how to see it by plotting numbers inside the network.
4. **Part 4 (backprop by hand):** compute gradients manually, once, so you never fear them again.
5. **Part 5 (WaveNet-style):** stacking layers into a deeper structure.

You can do parts 1-3 and return to 4-5 later if energy runs low.

Step 3: Let's build GPT and the Tokenizer

You have already watched the first one. Now do it properly: type along with the code, then change one thing (for example the number of layers) and watch what happens to the loss.

Then watch **"Let's build the GPT Tokenizer"**. A *tokenizer* converts text into numbers the model can read. Computers cannot read letters, only numbers, so "hello world" becomes something like `[31373, 995]`. BPE (byte pair encoding) is the method that decides how to chop text into pieces.

Checkpoint before moving on

You should be able to answer these without looking:

- What is a loss, and why do we want it to go down?
- What is a gradient?
- What does the model output at each step? (A probability for every possible next token.)
- Why do we turn text into numbers first?

Stage 1: A real base model (2-3 weeks)

What this stage is for: build and train a real transformer on real data, and learn the engineering tricks that make training fast and stable. At the end you own a model that writes readable text, and you understand every line that made it.

Key idea: the model from the video is a toy because it saw about 1 MB of text. Real models see gigabytes. The code barely changes. What changes is the data, the tokenizer, the size, and the training tricks.

Choose your path

Path	Model size	Data	Time and cost	Pick it if
A: TinyStories	20-50M parameters	TinyStories (simple children's stories)	1-3 hours on one free or cheap GPU	You want a quick win and coherent text soon
B: GPT-2 124M	124M parameters	FineWeb or OpenWebText (internet text)	About $10-20 on rented 8×A100, or several days on one good GPU	You want the full real pipeline

Do path A first. It is cheap, it finishes, and it teaches you the whole loop. Then do path B.

Path A: TinyStories, step by step

1. Download the TinyStories dataset from Hugging Face (huggingface.co/datasets/roneneldan/TinyStories). It is plain text.
2. Take your code from the "Let's build GPT" video. Replace the character tokenizer with the GPT-2 tokenizer: `pip install tiktoken`, then `enc = tiktoken.get_encoding("gpt2")`, and `enc.encode(text)` gives you a list of token numbers.
3. Convert the whole dataset to tokens once, save it as a file of numbers, and load it from disk during training. Do not re-tokenize every run.
4. Set the model to about 6 layers, 6 attention heads, and 384 embedding size, with context length 256. Increase later if your GPU allows.
5. Hold out 5% of the data as a *validation set*. Training loss tells you how well the model fits data it has seen. Validation loss tells you how well it handles data it has not seen. If training loss keeps falling while validation loss rises, the model is memorizing (*overfitting*).
6. Train. Every few hundred steps, print both losses and generate a sample of text. Watch the samples change from nonsense to words to sentences to stories.
7. **Done when:** the model writes short, grammatical stories with characters and a plot, and validation loss has flattened.

Path B: reproduce GPT-2 124M

Follow Karpathy's video **"Let's reproduce GPT-2 (124M)"** and his repo **nanoGPT**. Do it in this order so you never get lost:

1. Get the video model working with the GPT-2 tokenizer and architecture (12 layers, 12 heads, 768 embedding size, context length 1024).
2. Add the speed-ups one at a time, and measure tokens per second after each: mixed precision (`bfloat16` , uses half the memory), `torch.compile` (the library optimizes your code), and Flash Attention (`F.scaled_dot_product_attention` , a much faster attention).
3. Add the stability tricks: AdamW optimizer with weight decay, a learning rate that warms up then slowly decays (cosine schedule), and gradient clipping (stops one bad step from wrecking training).
4. Add gradient accumulation: run several small batches, add their gradients, then take one step. This imitates a huge batch on a small GPU.
5. Switch the data to FineWeb-Edu (a cleaned slice of the internet) and train for one pass.
6. Evaluate with HellaSwag (a multiple-choice test of common sense). GPT-2 124M scores about 29%.

Common beginner mistakes

- **Loss does not go down:** lower the learning rate, and check that your inputs and targets are shifted by one token.
- **Out of memory:** lower the batch size, lower the context length, or use gradient accumulation.
- **Loss starts around 10.8 for GPT-2:** this is correct. Random guessing over 50,257 tokens gives a loss of ln(50257), which is about 10.8.
- **Great training loss but gibberish samples:** check that you switch the model to evaluation mode (`model.eval()`) before generating.

Done when: you can explain what each trick does and why you added it, and your validation loss matches or beats published numbers for your model size.

Stage 2: Make it chat (2-3 weeks)

What this stage is for: your model from stage 1 only *continues text*. Ask it a question and it may continue with more questions. Post-training teaches it to behave like an assistant: you write a message, it writes a reply.

Key idea: a chat is just text with markers. The model does not know what a "conversation" is. You show it thousands of examples shaped like a conversation, and it learns to produce the assistant's half.

Step 1: Design a chat template

A *chat template* is the format that wraps each message. Add special tokens to your tokenizer so the model can tell who is speaking:

```
<|user|> What is the capital of France? <|end|>
<|assistant|> The capital of France is Paris. <|end|>
```

1. Add the new tokens (`<|user|>`, `<|assistant|>`, `<|end|>`) to your vocabulary. They get new ids at the end of the list.
2. The new tokens need new embedding rows. Make the embedding table larger, and initialize the new rows randomly.
3. Write one function `format_chat(messages)` that turns a list of messages into one string with these tokens. Use it for both training and chatting, so they never disagree.

Step 2: Supervised fine-tuning (SFT)

SFT means: continue training your stage-1 model, on conversation examples, with a lower learning rate.

1. Download a conversation dataset from Hugging Face: **SmolTalk** (good quality, small), UltraChat, or OpenAssistant. Start with a few thousand examples.
2. Format each conversation with your template and tokenize it.
3. **Loss masking:** compute the loss only on the assistant's tokens. Set the target for user tokens to `-100`, which PyTorch's `cross_entropy` ignores. Without this, the model wastes effort learning to imitate users.
4. Use a learning rate about 10 times lower than pretraining (for example 3e-5 instead of 3e-4), and train for 1-3 passes over the data. More passes make it memorize.
5. After training, test it: build the prompt with `format_chat`, end it with `<|assistant|>`, and let the model generate until it produces `<|end|>`.

Step 3: Write your own generation loop

This is the code that runs when you chat. Write it yourself once:

1. Encode the prompt into token ids.
2. Run the model to get a probability for every possible next token.
3. **Temperature:** divide the model's scores by a number before turning them into probabilities. Low (0.2) makes it safe and repetitive, high (1.2) makes it creative and risky.
4. **Top-p (nucleus sampling):** keep only the smallest group of tokens whose probabilities add up to, say, 0.9. Pick randomly among them.
5. Append the chosen token and repeat. Stop at `<|end|>` or at a maximum length.
6. **KV cache (do this last):** naive generation recomputes everything for every new token. A KV cache stores the earlier attention results so each step only computes the new token. It makes chat many times faster.
7. Wrap it in a simple `while True: input()` loop in the terminal. You now have a chatbot.

Step 4 (optional): Preference tuning with DPO

SFT teaches format. Preference tuning teaches *which answers are better*. The data is pairs: a prompt, a good answer ("chosen"), and a worse answer ("rejected"). DPO (Direct Preference Optimization) is a short loss function that makes the model more likely to produce the chosen answer than the rejected one, relative to a frozen copy of itself. Read the DPO paper's loss, then implement it in about 50 lines. Use a dataset like UltraFeedback.

Shortcut and what to expect

Shortcut: Karpathy's **nanochat** repo does the full pipeline (tokenizer, pretraining, SFT, reinforcement learning, a web chat page) in a compact codebase. Read it after you have built your own version, and compare the choices.

Expect: a small model that follows the chat format and answers simple questions, but makes things up often. That is normal at this size.

Done when: you can chat with your model in the terminal, and you can explain loss masking, temperature, and top-p without notes.

Stage 3: Coding model (3-4 weeks)

What this stage is for: turn your chat model into one that reads and writes code. Code is special text: it has strict rules, and you can *check* whether it is right by running it. That checkability is why coding is the best playground for AI.

Key idea: a coding model is a language model that saw a lot of code. Two extra tricks make it practical: fill-in-the-middle (so it can edit in the middle of a file) and tests (so you can measure it honestly).

Step 1: Get code data

1. Use a code dataset from Hugging Face. For a small budget, start with `the-stack-smol` (a small sample of The Stack, many languages). Choose one language first, for example Python, to keep it simple.
2. Check the license column. Keep only permissively licensed code (MIT, Apache, BSD) if you plan to share your model.
3. Clean it: remove huge files, files with very long lines, and exact duplicates. Duplicates make the model memorize.
4. Mix in about 10-20% normal English text and your chat data. Otherwise the model forgets how to talk.
5. Train with the same loop as stage 1, starting from your stage-1 weights (*continued pretraining*), with a lower learning rate. Tokenizer note: code has lots of spaces and symbols, so check that your tokenizer handles indentation efficiently. If it does not, train a new tokenizer on code (see the tokenizer video).

Step 2: Teach fill-in-the-middle (FIM)

Normal models only continue at the end. Your editor needs the model to fill a gap *between* existing code. The trick is a data rearrangement:

1. Take a code file and cut it randomly into three pieces: **prefix**, **middle**, **suffix**.
2. Add three new special tokens: `<|fim_prefix|>`, `<|fim_suffix|>`, `<|fim_middle|>`.
3. Rewrite the file as: prefix token, prefix text, suffix token, suffix text, middle token, **middle text**.
4. Do this to about half of your training files. Train normally. The model learns that after `<|fim_middle|>` it must produce the missing piece.
5. At use time, you give it the code before and after your cursor in that format, and it writes what goes between.

Step 3: Teach it to follow coding instructions

1. Use an instruction dataset such as Magicoder (OSS-Instruct), CodeAlpaca, or Evol-Instruct-Code. Each example is a coding request and a good answer.
2. Run SFT exactly as in stage 2 (chat template, loss masking, low learning rate).
3. Test by hand: ask for "a function that reverses a string", "a function that checks if a number is prime", and "explain this error message". Read the outputs critically.

Step 4: Measure honestly with tests

Reading outputs is not enough. Build an evaluation harness:

1. Download **HumanEval** (164 small Python problems, each with a hidden test) and **MBPP**.
2. For each problem, give the prompt to your model and collect the code it writes.
3. Run the code against the tests in a *sandbox*: a separate process with a time limit, or a Docker container with no network. Never run model-written code directly on your main system.
4. Record pass or fail for each problem.
5. **pass@1** is the fraction solved on the first try. **pass@k** is the chance that at least one of k tries passes. Generate several samples per problem to estimate it.
6. Re-run after every change. Now you can say "this change improved pass@1 from 3% to 5%" instead of "it feels better".

Step 5: Reality check with a pretrained model and LoRA

Your hobby model will score low. That is expected, and it is useful to see what good looks like:

1. Download a small open coding model, for example **Qwen2.5-Coder 1.5B**, from Hugging Face. Run your harness on it. Compare its score to yours.
2. Fine-tune it with **LoRA**. Full fine-tuning updates every weight, which needs lots of memory. LoRA freezes the original weights and adds two small matrices next to each big one, and trains only those. It fits on a single 24 GB GPU.
3. Write the LoRA layer yourself once (about 20 lines): output = original layer output + (input × A × B) × scale, where A and B are small matrices. Then compare with the `peft` library.
4. Fine-tune on a small coding dataset, re-run your harness, and see the score move.

Done when: you have a working harness that gives a pass@1 number, you can explain FIM and LoRA, and you have compared your model against a strong pretrained one.

Stage 4: Agentic (3-4 weeks)

What this stage is for: a chat model only *talks*. An agent *acts*: it can read your files, run commands, run tests, see the results, and try again. This stage is mostly ordinary programming around the model, not new neural network architecture.

Key idea: an agent is a loop. The model writes a request for a tool ("run this command"). Your program runs it and pastes the result back into the conversation. The model reads the result and decides the next step. Repeat until the task is done.

Step 1: Build the loop by hand

Use any capable model first (a hosted API or a strong local model), so you can focus on the loop. The core is short:

```
messages = [system_prompt, user_task]
for step in range(max_steps):
    reply = model(messages)             # 1. model answers
    messages.append(reply)
    call = parse_tool_call(reply)       # 2. did it ask for a tool?
    if call is None:
        break                           # no tool = it is finished
    result = run_tool(call)             # 3. YOUR code runs the tool
    messages.append(tool_result(result))# 4. show the model what happened
```

1. Decide a tool-call format. The simplest is JSON: `{"tool": "read_file", "path": "main.py"}`.
2. Write the tools as plain Python functions: `read_file(path)`, `write_file(path, text)`, `run_command(cmd)`, `run_tests()`. Each returns text.
3. Write `parse_tool_call`. Models sometimes produce broken JSON, so catch the error and tell the model "your last message was not valid JSON, try again".
4. Always set `max_steps` so a confused agent cannot loop forever.
5. Test with a tiny task: "Create hello.py that prints the numbers 1 to 5, then run it and confirm the output."

Step 2: Make it safe

An agent that runs commands can delete your files. Protect yourself from the first day:

1. Run tools inside a **Docker container** with only a project folder mounted and no network access.
2. Add a time limit to every command (for example 30 seconds).
3. Block or ask for confirmation on dangerous commands (`rm -rf` , `sudo` , anything touching folders outside the project).
4. Keep your agent in a copy of the project under Git, so you can undo anything with `git checkout` .

Step 3: Learn the standard patterns

- **ReAct:** the model writes a short *thought*, then an *action*, then reads the *observation*. Read the ReAct paper (short).
- **Plan, then act:** the model first writes a numbered plan, then executes it step by step, updating the plan when something fails.
- **Error recovery:** when a command fails, the error text goes back to the model, which fixes the code and retries. Most of an agent's value comes from this.
- **Context management:** the conversation grows with every step and will exceed the model's context length. Shorten old tool outputs (keep the first and last lines), or summarize old steps.

Step 4: Make your own small model use tools

1. Run a strong model through your agent loop on many small tasks (write a function, fix a failing test). Save every full conversation.
2. Keep only the runs where the tests passed at the end. This is *rejection sampling*: reject the failures, keep the successes.
3. Turn the kept runs into SFT data (as in stage 2), with the loss on the assistant's turns only.
4. Fine-tune your small model on it. Then run the same tasks with your model in the loop and compare success rates.

Step 5: Reinforcement learning with verifiable rewards

This is how modern coding agents keep improving. The idea in plain words: let the model attempt a task many times, check each attempt with tests (reward 1 for pass, 0 for fail), and make passing attempts more likely.

1. Build a set of 100-500 small tasks that each have an automatic pass or fail test.
2. Start simple: sample 8 attempts per task, keep the passing ones, fine-tune on them, repeat. This is the rejection-sampling loop again, run several rounds.
3. Later, learn **GRPO** (the method from the DeepSeek-R1 paper): for each task, compare the attempts against each other, and push the model toward the ones that did better than average.
4. Watch out for *reward hacking*: a model may learn to cheat, for example by editing the test file. Keep tests read-only and hidden from the agent.

Step 6: Evaluate

Collect 20-50 realistic tasks from your own projects ("add this feature", "fix this bug"), each with a test. Run your agent on them before and after every change, and record the pass rate. Public benchmarks like SWE-bench show how professional agents are measured, but are too heavy for a first project.

Done when: your agent completes small coding tasks end to end inside a sandbox, and you can show a measured improvement from fine-tuning on its own successful runs.

Compute and cost

Stage	Hardware	Approximate cost
0-1 (small models)	Single consumer GPU or a Colab session	Low
1 (GPT-2 124M reproduction)	Rented 8×A100, or a few days on one good GPU	$10-20
2-4 (fine-tuning small models)	One 24GB GPU, owned or rented	A few dollars per hour rented

Tips for learning at the lowest level

Understanding comes from building each piece yourself, then checking it against a trusted version.

1. **Write first, compare second.** Write each component yourself (attention, optimizer, tokenizer, LoRA layer, DPO loss). Then compare your output against the PyTorch or Hugging Face version on the same input. If the numbers match, you understood it. If not, you found a bug or a gap.
2. **Overfit one tiny batch first.** Before any long run, train on just 4-8 examples. The loss should drop to nearly zero. If it does not, you have a bug, and you found it in one minute instead of after six hours.
3. **Keep an experiment log.** For every run, write down: what you changed, the settings, the final loss, and one sentence on what you learned. Future you will thank you.
4. **Know what a normal loss looks like.** It starts near ln(vocabulary size), falls quickly, then slowly flattens. Spikes, flat lines, or `nan` mean something is wrong (usually the learning rate).
5. **Read code in a good order.** nanoGPT first (small and clear), then nanochat (the full pipeline), then a production library such as Hugging Face `transformers` or `torchtune`.
6. **Ask for help well.** When posting a question, include your code, the full error, and what you already tried. Good places: the Hugging Face forums, the r/LocalLLaMA and r/MachineLearning subreddits, and the Discord communities around Karpathy's and Hugging Face's projects.

When you feel lost

Go one level down. If a trick in a video confuses you, write the smallest possible example with 3 numbers and print every step. If a paper confuses you, find a blog post or video that explains it, then go back to the paper.

Papers worth reading along the way

Reading papers is a skill you build slowly. Read the abstract, the figures, and the method section first. Skip the proofs on your first pass.

When	Paper	What it teaches
Stage 1	*Attention Is All You Need*	The transformer architecture

When	Paper	What it teaches
Stage 1	GPT-2 and GPT-3 papers	Scaling a language model
Stage 1	Chinchilla ("Training Compute-Optimal LLMs")	How much data per parameter
Stage 2	InstructGPT	How chat behavior is trained (RLHF)
Stage 2	DPO	A simpler way to learn from preferences
Stage 3	LoRA	Cheap fine-tuning with small matrices
Stage 3	StarCoder	How a real code model is built
Stage 4	ReAct	Thought, action, observation loops
Stage 4	DeepSeek-R1	Reinforcement learning with GRPO

Glossary (plain English)

Term	Meaning
Token	A piece of text (a word, part of a word, or a symbol) turned into a number the model can read
Tokenizer	The program that converts text to tokens and back
BPE	Byte pair encoding: the method that decides which pieces of text become tokens
Embedding	A list of numbers that represents a token; similar meanings get similar numbers
Parameter / weight	One number inside the model that training adjusts. "124M parameters" means 124 million of them

Term	Meaning
Transformer	The neural network design behind GPT. It uses attention to let each token look at earlier tokens
Attention	The step where each token decides which earlier tokens matter for predicting what comes next
Context length	How many tokens the model can look at at once
Loss	A score of how wrong the model is. Training tries to make it smaller
Backpropagation	Computing how much each weight contributed to the loss, by working backwards through the network
Gradient descent	Nudging every weight a little in the direction that lowers the loss
Learning rate	How big each nudge is. Too big breaks training, too small is slow
Batch	A group of examples processed together in one step
Epoch	One full pass over the training data
Overfitting	The model memorizes training data and does badly on new data
Validation set	Data held back from training, used to check for overfitting
Pretraining	Training on huge amounts of raw text to learn language
SFT	Supervised fine-tuning: training on examples of the behavior you want, such as chat replies
RLHF / DPO	Ways to teach the model which answers people prefer
LoRA	Fine-tuning that trains small added matrices instead of all weights

Term	Meaning
FIM	Fill-in-the-middle: training a model to write code between a prefix and a suffix
KV cache	Stored attention results that make text generation faster
Temperature / top-p	Settings that control how random or safe the next-token choice is
pass@k	The chance that at least one of k generated solutions passes the tests
Agent	A model in a loop that can call tools, see results, and decide what to do next
Sandbox	An isolated environment where code runs without being able to harm your computer
GRPO	A reinforcement learning method that compares several attempts at one task and favors the better ones